Operating Systems - Numerical Practice Problems for GATE

1. CPU Scheduling Algorithms - Practice Problems

Problem 1: FCFS (First Come First Serve)
Given processes with arrival time and burst time:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Find: Completion Time, Turnaround Time, Waiting Time, Average WT, Average TAT

Solution:
```
Gantt Chart: |P1(0-8)|P2(8-12)|P3(12-21)|P4(21-26)|

Process | AT | BT | CT | TAT(CT-AT) | WT(TAT-BT)
P1      | 0  | 8  | 8  | 8          | 0
P2      | 1  | 4  | 12 | 11         | 7
P3      | 2  | 9  | 21 | 19         | 10
P4      | 3  | 5  | 26 | 23         | 18

Average TAT = (8+11+19+23)/4 = 15.25
Average WT = (0+7+10+18)/4 = 8.75
```

Problem 2: SJF Non-Preemptive
Given processes:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Find: Average Waiting Time and Average Turnaround Time

Solution:
```
At time 0: Only P1 available → P1 runs (0-7)
At time 7: P2, P3, P4 available. P3 has shortest BT → P3 runs (7-8)
At time 8: P2, P4 available. Both have BT=4, P2 arrived first → P2 runs (8-12)
At time 12: Only P4 left → P4 runs (12-16)
```

Gantt Chart: |P1(0-7)|P3(7-8)|P2(8-12)|P4(12-16)|

Process | AT | BT | CT | TAT | WT
P1 | 0 | 7 | 7 | 7 | 0
P2 | 2 | 4 | 12 | 10 | 6
P3 | 4 | 1 | 8 | 4 | 3
P4 | 5 | 4 | 16 | 11 | 7

Average TAT = (7+10+4+11)/4 = 8
Average WT = (0+6+3+7)/4 = 4
```

### Problem 3: SRTF (Shortest Remaining Time First)
**Given processes:**

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 1 | 4 |
| P3 | 2 | 9 |
| P4 | 3 | 5 |

**Find:** Average Waiting Time

**Solution:**
```
Time 0-1: P1 runs (remaining = 7)
Time 1: P2 arrives (BT=4 < P1's remaining=7) → P2 preempts P1
Time 1-2: P2 runs (remaining = 3)
Time 2: P3 arrives (BT=9 > P2's remaining=3) → P2 continues
Time 2-3: P2 runs (remaining = 2)
Time 3: P4 arrives (BT=5 > P2's remaining=2) → P2 continues
Time 3-5: P2 runs and completes
Time 5: Available: P1(remaining=7), P3(BT=9), P4(BT=5)
      P4 has shortest → P4 runs (5-10)
Time 10: P1(remaining=7) < P3(BT=9) → P1 runs (10-17)
Time 17: P3 runs (17-26)

Gantt Chart: |P1(0-1)|P2(1-5)|P4(5-10)|P1(10-17)|P3(17-26)|

Process | AT | BT | CT | TAT | WT
P1 | 0 | 8 | 17 | 17 | 9
P2 | 1 | 4 | 5 | 4 | 0
P3 | 2 | 9 | 26 | 24 | 15
P4 | 3 | 5 | 10 | 7 | 2

Average WT = (9+0+15+2)/4 = 6.5
```
```

```

### Problem 4: Round Robin
**Given processes with Time Quantum = 2:**

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 5          |
| P2      | 1            | 3          |
| P3      | 2            | 1          |
| P4      | 3            | 2          |

**Find:** Average Response Time and Average Turnaround Time

**Solution:**
```
Time 0-2: P1 runs (remaining = 3), Response Time of P1 = 0
Time 2-3: P2 runs (remaining = 2), Response Time of P2 = 2-1 = 1
Time 3-4: P3 runs and completes, Response Time of P3 = 3-2 = 1
Time 4-6: P4 runs and completes, Response Time of P4 = 4-3 = 1
Time 6-8: P1 runs (remaining = 1)
Time 8-9: P2 runs (remaining = 1)
Time 9-10: P1 runs and completes
Time 10-11: P2 runs and completes

Gantt Chart: |P1(0-2)|P2(2-3)|P3(3-4)|P4(4-6)|P1(6-8)|P2(8-9)|P1(9-10)|P2(10-11)|

Process | AT | BT | CT | TAT | RT
P1 | 0 | 5 | 10 | 10 | 0
P2 | 1 | 3 | 11 | 10 | 1
P3 | 2 | 1 | 4 | 2 | 1
P4 | 3 | 2 | 6 | 3 | 1

Average Response Time = (0+1+1+1)/4 = 0.75
Average TAT = (10+10+2+3)/4 = 6.25
```

---

## 2. Address Translation in Paging - Practice Problems

### Problem 5: Basic Paging
**System Configuration:**
- Logical Address Space: 16-bit (64K)
- Page Size: 4K (4096 bytes)
- Physical Address Space: 18-bit (256K)

**Questions:**
```

1. How many pages in logical address space?
2. How many frames in physical address space?
3. How many bits for page number and offset?
4. Translate logical address 20500

Solution:
```
1. Number of pages = 64K / 4K = 16 pages
2. Number of frames = 256K / 4K = 64 frames
3. Page offset bits = log₂(4096) = 12 bits
   Page number bits = 16 - 12 = 4 bits
4. Logical Address 20500:
   20500 = 5 × 4096 + 20 (Page 5, Offset 20)
   Binary: Page=0101, Offset=000000000010100

   If Page 5 maps to Frame 12:
   Physical Address = 12 × 4096 + 20 = 49172
```

Problem 6: Page Table Translation
Given:
- Page Size: 1K (1024 bytes)
- Page Table for Process P:

Page #	Frame #	Valid
0	3	1
1	7	1
2	1	1
3	4	0

Translate these logical addresses:
1. 1500
2. 3000
3. 2560

Solution:
```
Page size = 1024 bytes, so offset = 10 bits

1. Logical Address 1500:
   Page = 1500 / 1024 = 1, Offset = 1500 % 1024 = 476
   Page 1 → Frame 7, Valid = 1
   Physical Address = 7 × 1024 + 476 = 7644

2. Logical Address 3000:
   Page = 3000 / 1024 = 2, Offset = 3000 % 1024 = 952
   Page 2 → Frame 1, Valid = 1
```

Physical Address = 1 × 1024 + 952 = 1976

3. Logical Address 2560:
Page = 2560 / 1024 = 2, Offset = 2560 % 1024 = 512
Page 2 → Frame 1, Valid = 1
Physical Address = 1 × 1024 + 512 = 1536
```

### Problem 7: Two-Level Paging
**System:**
- 32-bit logical address
- Page size: 4K
- Page table entry: 4 bytes
- Memory frame size for page table: 4K

**Find:** Structure of logical address and maximum logical address space

**Solution:**
```
Page size = 4K = 2^{12} bytes
Page offset = 12 bits

Entries per page table page = 4K / 4 bytes = 1K = 2^{10} entries
Second-level page number = 10 bits

Remaining bits for first-level = 32 - 12 - 10 = 10 bits
First-level page number = 10 bits

Logical Address Structure:
|First-level page(10)|Second-level page(10)|Offset(12)|

Maximum pages = 2^{10} × 2^{10} = 2^{20} pages
Maximum logical address space = 2^{20} × 4K = 4GB
```

---

## 3. Page Replacement Algorithms - Practice Problems

### Problem 8: FIFO Page Replacement
**Given:**
- Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of frames: 3

**Find:** Number of page faults using FIFO

**Solution:**
```

Frame Status:
Step | Ref | Frame1 | Frame2 | Frame3 | Fault?
1 | 1 | 1 | - | - | Yes
2 | 2 | 1 | 2 | - | Yes
3 | 3 | 1 | 2 | 3 | Yes
4 | 4 | 4 | 2 | 3 | Yes (replace 1)
5 | 1 | 4 | 1 | 3 | Yes (replace 2)
6 | 2 | 4 | 1 | 2 | Yes (replace 3)
7 | 5 | 5 | 1 | 2 | Yes (replace 4)
8 | 1 | 5 | 1 | 2 | No
9 | 2 | 5 | 1 | 2 | No
10 | 3 | 3 | 1 | 2 | Yes (replace 5)
11 | 4 | 3 | 4 | 2 | Yes (replace 1)
12 | 5 | 3 | 4 | 5 | Yes (replace 2)

Total Page Faults = 10
```

### Problem 9: LRU Page Replacement
**Given:**
- Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
- Number of frames: 4

**Find:** Number of page faults using LRU

**Solution:**
```
Using LRU (track last used time):

Step | Ref | Frame Status | Fault? | LRU Info
1 | 7 | [7, -, -, -] | Yes | 7 used at time 1
2 | 0 | [7, 0, -, -] | Yes | 0 used at time 2
3 | 1 | [7, 0, 1, -] | Yes | 1 used at time 3
4 | 2 | [7, 0, 1, 2] | Yes | 2 used at time 4
5 | 0 | [7, 0, 1, 2] | No | 0 used at time 5
6 | 3 | [3, 0, 1, 2] | Yes | Replace 7 (LRU)
7 | 0 | [3, 0, 1, 2] | No | 0 used at time 7
8 | 4 | [3, 0, 4, 2] | Yes | Replace 1 (LRU)
9 | 2 | [3, 0, 4, 2] | No | 2 used at time 9
10 | 3 | [3, 0, 4, 2] | No | 3 used at time 10
11 | 0 | [3, 0, 4, 2] | No | 0 used at time 11
12 | 3 | [3, 0, 4, 2] | No | 3 used at time 12
13 | 2 | [3, 0, 4, 2] | No | 2 used at time 13

Total Page Faults = 6
```

### Problem 10: Optimal Page Replacement
```

Given:
- Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Number of frames: 3

Find: Number of page faults using Optimal algorithm

Solution:
```
Optimal: Replace page that will be used farthest in future

Step | Ref | Frames    | Fault? | Future References
1    | 1   | [1,-,-]   | Yes    | 1 next at pos 5
2    | 2   | [1,2,-]   | Yes    | 2 next at pos 6
3    | 3   | [1,2,3]   | Yes    | 3 next at pos 10
4    | 4   | [1,2,4]   | Yes    | Replace 3 (farthest future use)
5    | 1   | [1,2,4]   | No     |
6    | 2   | [1,2,4]   | No     |
7    | 5   | [1,2,5]   | Yes    | Replace 4 (farthest future use)
8    | 1   | [1,2,5]   | No     |
9    | 2   | [1,2,5]   | No     |
10   | 3   | [1,3,5]   | Yes    | Replace 2 (no more future refs)
11   | 4   | [4,3,5]   | Yes    | Replace 1 (no more future refs)
12   | 5   | [4,3,5]   | No     |

Total Page Faults = 7 (This is optimal)
```

Practice Tips for GATE:

CPU Scheduling:
1. Always draw Gantt chart first
2. For preemptive algorithms, check at every arrival time
3. Remember: Average WT = (Sum of individual WT) / Number of processes
4. SRTF always gives minimum average waiting time

Address Translation:
1. **Key Formula:** Page Number = Logical Address / Page Size
2. **Offset:** Logical Address % Page Size
3. **Physical Address:** Frame Number × Page Size + Offset
4. Always check if page is valid before translation

Page Replacement:
1. **FIFO:** Simple queue, can have Belady's anomaly
2. **LRU:** Track usage time, practical and good performance
3. **Optimal:** Theoretical minimum, look ahead in reference string

4. **Remember:** More frames generally mean fewer page faults (except Belady's anomaly in FIFO)

Quick Formulas to Remember:
- **Page Offset bits** = $\log_2$(Page Size)
- **Number of pages** = Logical Address Space / Page Size
- **Number of frames** = Physical Address Space / Frame Size
- **Effective Access Time** = Hit Rate × Memory Time + Miss Rate × (Memory Time + Page Fault Time)

Try solving these problems step by step. The key to GATE success is practicing similar numerical problems until the approach becomes automatic!